CS 344: Design and Analysis of Computer Algorithms **Rutgers: Spring 2026**

Final Exam

Name: *Netid:*

Instructions

1. Write your name and NetID at the top of the exam.
2. The exam consists of 5 problems worth 150 points in total, plus one extra credit problem worth 20 points.
3. This is an open-notes exam. You may consult any course materials (e.g., lecture notes, textbook), but no other resources are allowed.
4. Provide proofs of correctness for your algorithms and analyze their runtime.
5. Avoid overly complicated pseudo-code; explain your algorithm clearly in English.
6. You may use any algorithm presented in class or homework as a building block for your solutions.

Problem #	**Points**	**Score**
1	30	
2	30	
3	30	
4	30	
5	30	
6	+20	
Total	**150+20**	

---

1. (a) For the following weighted graph

   i. Determine the BFS traversal starting from A.
   ii. Determine DFS traversal starting from A.
   iii. Find the strongly connected components. Is it acyclic?
   iv. Find a minimum spanning tree.
   v. Find the shortest path from A to F.
   vi. Find the shortest path from from A to F using at most 2 edges.
   vii. Find the shortest path using from A to F using only B, C, D, E as intermediate vertices.
   viii. Find the maximum flow from A to F if the edge weight are capacities of the network.

   Please show all the work.

   `graph.png`

   i. BFS from A: If the adjacent vertices are maintained in alphabetic order we get:

$$A, D, E, G, F, C, B$$

   ii. DFS from A: If the adjacent vertices are maintained in alphabetic order we get:

$$A, D, G, C, E, F, B$$

iii. It is not acyclic. There is a directed cycle from E to G back to E.
The scc components are

$$\{A\}, \{D\}, \{C, E, G\}, \{B\}, \{F\}$$

iv. The Prim's algorithm starting with A gives the tree:

$$A - D, B - A, A - G, G - E, B - F, C - F$$

(There are other solution possible based ont the specifics of the implementation/ the choices made for the "cheapest" edge when there are multiple options)

v. Dikjstra's on the graph gives

$$A - D, A - G, G - E, E - F, G - C$$

The shortest path from A to F therefore is $A - G - E - F$.

vi. Run two iterations of Bellman-Ford. You can see that the distance from $d_A[F]$ is $+\infty$, hence there is not path with at most two edges.

vii. Run Floyd-Warshall, each iteration adding intermediate vertices B, C, D, E.
The shortest path is $A - E - F$.

viii. Run Ford-Fulkerson:
The maximum flow is 3, push 2 units from $A - E - F$ and then one unit $A - E - G - C - F$.
At this point no augmenting path exists. In fact the cut $\{A, D, E, G, C\}, \{F, B\}$ has capacity of $1 + 2 = 3$.

(b) Every question below has one of the following answers. (a) $\Theta(1)$ (b) $\Theta(\log n)$ (c) $\Theta(n)$ (d) $\Theta(n \log n)$ (e) $\Theta\left(n^2\right)$

i. What is the solution of the recurrence $T(n) = T(\sqrt{n}) + n$?

ii. What is the solution of the recurrence $T(n) = T(n-1) + \lg n$?

iii. What is the solution of the recurrence $T(n) = 2T\left(\left\lceil \frac{n+27}{2} \right\rceil\right) + 5n - 7\sqrt{\lg n} + \frac{1999}{n}$?

iv. What is $\sum_{i=1}^{n} i$?

v. What is $\sum_{i=1}^{n} \frac{1}{i}$?

vi. How many bits are there in the binary representation of the nth Fibonacci number?

vii. What is the worst-case running time of randomized quicksort?

viii. How many digits do you need to write $n!$ in decimal?

ix. What is $\sum_{i=1}^{\lg n} 2^i$?

x. What is $f(n) = e^{\log \log \sqrt{n} + \log(n-1)}$?

Solution:

i. Write $n = 2^m$ and $T(n) = S(m)$, we get $S(m) = S(m/2) + 2^m$, $S(m) = \theta(2^m)$, so $T(n) = \Theta(n)$.

ii. We get $T(n) = \log n + \log(n-1) + ... = \Theta(n \log n)$

iii. It reduces to the recurrence $T(n) = 2T(n/2) + \Theta(n)$, hence we get $T(n) = \Theta(n \log n)$.

iv. $\sum_{i=1}^{n} i = \frac{n(n+1}{2} = \Theta(n^2)$

v. $\sum_{i=1}^{n} \frac{1}{i}$ is approximately $\int_1^n \frac{1}{x} dx = \Theta(\log n)$

vi. Fibonacci numbers F_n growly exponentially $\sim \phi^n, \phi \approx 1.618$, so the number of bit need to represent F_n is: $\approx \log \phi^n = \Theta(n)$.

vii. In the worst case, the randomized quicksort takes $\Theta(n^2)$ time.

viii. The number of digits in $n!$ is $\log_{10} n! = \Theta(n \log n)$.

ix. $\sum_{i=1}^{\log_2 n} 2^i = \Theta(2^{\log_2 n}) = \Theta(n)$

x. $f(n) = e^{\log \log \sqrt{n} + \log(n-1)}$ is $\log \sqrt{n} \times (n-1) = \Theta(n \log n)$.

2. (a) Given a grid of integers, design a dynamic programming algorithm to find a path from the top-left to the bottom-right that minimizes the sum of the numbers along the path. Movement is allowed only down or right. Provide the specification, recursive solution, and a bottom-up DP solution.

   (b) For the given grid below, execute the algorithm and show the resulting DP table:

$$G = \begin{bmatrix} 1 & 3 & 1 \\ 1 & 5 & 1 \\ 4 & 2 & 1 \end{bmatrix}$$

   Clearly demonstrate how the DP table is filled and identify the minimum path sum.

   (c) If you are allowed to move left, right, up and down at any point in the grid, give an algorithm to determine if you can reach from top left to bottom right with a monochromatic path (path with the same number for every cell)

Specification For every $0 \leq i < m$ and $0 \leq j < n$, define:

- $F(i, j)$: the minimum sum of numbers along a path from the top-left corner $(0, 0)$ to the cell (i, j).

The solution to the problem is $F(m-1, n-1)$, where m and n are the dimensions of the grid.

Recursive Formula The recursive formula for $F(i, j)$ is as follows:

$$F(i,j) = \begin{cases} G(0,0), & \text{if } i = 0 \text{ and } j = 0, \\ F(i-1,j) + G(i,j), & \text{if } i > 0 \text{ and } j = 0, \\ F(i,j-1) + G(i,j), & \text{if } i = 0 \text{ and } j > 0, \\ \min(F(i-1,j), F(i,j-1)) + G(i,j), & \text{if } i > 0 \text{ and } j > 0. \end{cases}$$

Bottom-Up Dynamic Programming Solution

(a) Initialize a 2D DP table $T[0 : m-1][0 : n-1]$ of size $m \times n$ to store the values of $F(i, j)$.

(b) For each cell (i, j):
   - If $i = 0$ and $j = 0$, set $T[i][j] = G(0, 0)$.
   - If $i > 0$ and $j = 0$, set $T[i][j] = T[i-1][j] + G(i, j)$.
   - If $i = 0$ and $j > 0$, set $T[i][j] = T[i][j-1] + G(i, j)$.
   - Otherwise, set:
$$T[i][j] = \min(T[i-1][j], T[i][j-1]) + G(i, j).$$

(c) Return $T[m-1][n-1]$.

To ensure all dependencies are computed before updating a cell, the DP table is filled **row by row**, from **top to bottom**, and within each row, from **left to right**. The filling order guarantees that:

- $T[i-1][j]$ (the value from the cell above) is computed before $T[i][j]$.
- $T[i][j-1]$ (the value from the cell to the left) is computed before $T[i][j]$.

Filling Order for a Grid G For a grid G of size $m \times n$:

(a) Start at the top-left corner $(0, 0)$.

(b) Process all cells in the first row ($i = 0, j = 0$ to $n-1$).

(c) Process all cells in the first column ($j = 0, i = 1$ to $m-1$).

(d) For the remaining cells (i, j), process each row i, iterating through columns $j = 1$ to $n-1$.

(e) Repeat until reaching the bottom-right corner $(m-1, n-1)$.

b) Execution:

Given the grid:

$$G = \begin{bmatrix} 1 & 3 & 1 \\ 1 & 5 & 1 \\ 4 & 2 & 1 \end{bmatrix}$$

We initialize the DP table T and compute its values:

$T(0,0)$	$T(0,1)$	$T(0,2)$
$T(1,0)$	$T(1,1)$	$T(1,2)$
$T(2,0)$	$T(2,1)$	$T(2,2)$

Fill in the values step-by-step:

- Base case: $T(0,0) = G(0,0) = 1$.
- First row: $T(0,j) = T(0,j-1) + G(0,j)$.

$$T(0,1) = 1 + 3 = 4, \quad T(0,2) = 4 + 1 = 5.$$

- First column: $T(i,0) = T(i-1,0) + G(i,0)$.

$$T(1,0) = 1 + 1 = 2, \quad T(2,0) = 2 + 4 = 6.$$

- Remaining cells:

$$T(1,1) = \min(T(0,1), T(1,0)) + G(1,1) = \min(4,2) + 5 = 7.$$

$$T(1,2) = \min(T(0,2), T(1,1)) + G(1,2) = \min(5,7) + 1 = 6.$$

$$T(2,1) = \min(T(1,1), T(2,0)) + G(2,1) = \min(7,6) + 2 = 8.$$

$$T(2,2) = \min(T(1,2), T(2,1)) + G(2,2) = \min(6,8) + 1 = 7.$$

The final DP table is:

$$T = \begin{bmatrix} 1 & 4 & 5 \\ 2 & 7 & 6 \\ 6 & 8 & 7 \end{bmatrix}$$

The minimum path sum from the top-left to the bottom-right is $T(2,2) = 7$.

c) If you are allowed to move left, right, up and down at any point in the grid, give an algorithm to determine if you can reach from top left to bottom right with a monochromatic path (path with the same number for every cell)

Solution:

We reduce to this problem to graph search. Hence BFS/DFS can solve it in $O(E+V) = O(n^2)$, where n is the size of the grid.

Look at the element $G(0,0)$. Now consider the graph G with vertices given by the cells (i,j) of the grid and edges between two adjacent cells (i,j) , (k,l) (up/down, right/left) if both the the cells have contain the element $G(i,j) = G(k,l) = G(0,0)$.

Starting the graph search from $(0,0)$ and search the element $(m-1, n-1)$.

The number of vertices is $|V| = n^2$ and the edges is $|E| = O(n^2)$. Hence the algorithm runs in $O(n^2)$ time.

3. Given a connected, undirected graph $G = (V, E)$, where each edge $e \in E$ has a unique weight $w(e)$, consider the follow algorithm to find the minimum spanning tree (MST) by iteratively discarding edges.

Reverse-Delete Method

- Start with the entire graph $G = (V, E)$.
- Sort all edges in decreasing order of weight.
- For each edge in the sorted list, do the following:
  - Temporarily remove the edge from the graph.
  - Check if the graph remains connected.
  - If it does, permanently discard the edge; otherwise, retain it.
- Repeat until all edges have been processed.
- The remaining edges form the MST.

(a) Prove that the reverse-delete method results in an MST.

(b) If there a data structure in which deleting an edge, checking the connectivity of the resulting graph, and (if it is disconnected) re-inserting the edge can be done in $O(\log^3 V)$, what is the time complexity of the algorithm?

(c) Here is divide-and-conquer algorithm for computing minimum spanning trees, which goes as follows. Partition the set V of vertices into two sets V_1 and V_2 such that $|V_1|$ and $|V_2|$ differ by at most 1 . Let E_1 be the set of edges that are incident only on vertices in V_1, and let E_2 be the set of edges that are incident only on vertices in V_2. Recursively solve a minimum-spanning-tree problem on each of the two subgraphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$. Finally, select the minimum-weight edge in E that crosses the cut V_1, V_2, and use this edge to unite the resulting two minimum spanning trees into a single spanning tree. Prove that this algorithm doesn't work.

Solution:

(a) The Reverse-Delete algorithm produces a spanning tree:

- The algorithm ensures connectivity because edges are only removed if the graph remains connected.
- It eliminates cycles, as any edge removed eliminates a cycle without disconnecting the graph.

The resulting spanning tree is a minimum spanning tree:

- Consider any edge e removed by the algorithm. At the time of removal, e must lie on some cycle C (otherwise, its removal would disconnect the graph). e should be the most expensive edge in C, because otherwise some other edge in the cycle would be removed by the algorithm first.
- Let C be any cycle in G, and let $e = (v, w)$ be the most expensive edge in C. Then e does not belong to any minimum spanning tree (MST).
  i. Assume T is an MST that contains e.
  ii. Removing e from T partitions the vertices into two components:
     - S, which contains v, and
     - $V - S$, which contains w.
  iii. The remaining edges of C form a path P between v and w. There is at least one edge e' in P that connects S and $V - S$.
  iv. Adding e' to $T - e$ results in a new tree T', which is connected and contains no cycles (since e was the only edge creating a cycle).
  v. Since e is the most expensive edge in C, replacing e with e' reduces the total weight of the tree, contradicting the assumption that T is an MST.
  vi. Therefore, e cannot belong to any MST.

- The algorithm retains only edges that are necessary to maintain connectivity and discards edges that cannot belong to any MST, ensuring the resulting tree is minimum.

(b) **Total time complexity:**

- Sorting the edges takes $O(|E| \log |E|)$.
- Processing each of the $|E|$ edges takes $O(|E| \cdot \log^3 |V|)$.

Since $|E| \leq |V|^2$, we can simplify $\log |E| = O(\log |V|)$. The overall complexity is:

$$O(|E| \cdot \log^3 |V|)$$

(c) Consider the graph $G = (V, E)$, where:

$$V = \{a, b, c, d\}, \quad E = \{(a, b, 1), (b, c, 1), (c, d, 1), (a, d, 2)\}.$$

The numbers in parentheses denote the weights of the edges.

- Partition the vertices into $V_1 = \{a, b\}$ and $V_2 = \{c, d\}$.
- Solve the MSTs of $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$:

$$G_1 : T_1 = \{(a, b, 1)\}, \quad G_2 : T_2 = \{(c, d, 1)\}.$$

- Select the minimum-weight edge crossing the cut (V_1, V_2):

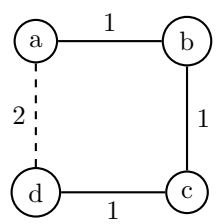
$$(b, c, 1).$$

- Combine T_1, T_2, and $(b, c, 1)$ to form:

$$T = \{(a, b, 1), (b, c, 1), (c, d, 1)\}.$$

However, the actual MST of G is:

$$T_{\text{MST}} = \{(a, b, 1), (b, c, 1), (c, d, 1)\}.$$



4. Alice is visiting a new country: Algorithmia. Within Algorithmia there are n cities. In addition, there are m roads and airline routes. Alice does not enjoy flying but does not mind driving at all. Design an algorithm that will tell Alice how to get from city s to t using a minimum number of airline routes, but, it can use any number of roads. Your algorithm should run in $O(m + n \log n)$ time. You can assume that both the roads and airline routes are two-way, namely, can be used in either direction.

   A complete solution consists of three parts: the algorithm or reduction, the proof of correctness, and the runtime analysis.

   Example: Suppose there are two paths between s and t, one with 5 airline routes plus 2 roads, and the other with 2 airline routes plus 20 roads. Then, Alice would like to take the second path.

   Solution: Reduction to Dijkstra's Algorithm

   To solve the problem of finding the path from city s to city t with the minimum number of airline routes, we reduce the problem to a variant of Dijkstra's algorithm as follows:

   - Represent the cities and connections as a graph $G = (V, E)$:
     - The cities are the vertices V.
     - The roads and airline routes are the edges E, with weights assigned as follows:
       * Roads have weight 0.
       * Airline routes have weight 1.
   - Use Dijkstra's algorithm with a Fibonacci heap to compute the shortest path from s to t in this graph, where the edge weights represent the number of airline routes used.
   - The shortest path computed by Dijkstra's algorithm corresponds to the path that minimizes the total number of airline routes used, as the algorithm inherently prioritizes paths with the smallest cumulative weight.

   Proof of correctness: Dijkstra's output the the path with minimum cost, which in this case corrsponds to a path with minimum airlines routes.

   Time Complexity: Dijkstra's algorithm with a Fibonacci heap runs in $O(E + V \log V)$, hence the this algorithm runs in $O(m + n \log n)$ time.

5. You need to determine if it's possible to assign at least one doctor to cover each vacation day, given the following:

   Input:

   - k vacation periods, each spanning days D_j, with $\bigcup_j D_j$ being the set of all vacation days.
   - n doctors, each available on specific days S_i.

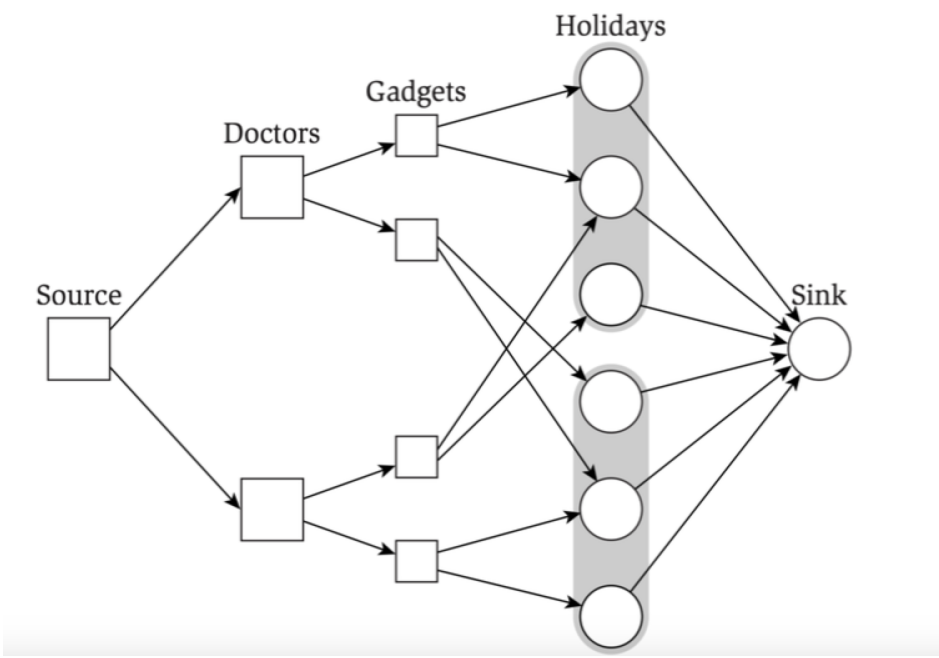
   Constraints:

   (a) Each doctor can work at most c vacation days total.

   (b) For each vacation period j, no doctor can work more than one day in D_j.

   (c) A doctor can only work on days they are available (S_i).

   Goal: Determine if every day in $\bigcup_j D_j$ can be covered under these constraints in polynomial time.



We reduce the problem to a **maximum flow problem** in a directed graph $G = (V, E)$.

Graph Construction

Nodes:

- s: Source node.
- t: Sink node.
- u_i: Doctor nodes ($i = 1, \ldots, n$).
- v_{ij}: Intermediate nodes for doctor i and vacation period j.
- d_k: Vacation day nodes ($k \in \bigcup_j D_j$).

Edges:

(a) From source to doctor nodes:

$$(s, u_i) \text{ with capacity } c, \quad \forall i = 1, \ldots, n.$$

(b) From doctor nodes to intermediate nodes:

$$(u_i, v_{ij}) \text{ with capacity } 1, \quad \forall i = 1, \ldots, n \text{ and } j \text{ such that } D_j \cap S_i \neq \emptyset.$$

(c) From intermediate nodes to vacation day nodes:

$$(v_{ij}, d_k) \text{ with capacity } 1, \quad \text{if } d_k \in D_j.$$

(d) From vacation day nodes to sink:

$$(d_k, t) \text{ with capacity } 1, \quad \forall k \in \bigcup_j D_j.$$

Flow Interpretation

- The goal is to determine if a flow of size $|\bigcup_j D_j|$ exists from s to t.
- If such a flow exists, it is possible to assign doctors to cover all vacation days while satisfying all constraints. If not, it is impossible.

Each unit of flow corresponds to a selection of a doctor, a vacation period and a holiday in that vacation period. The capacity of c for source to doctors will ensure that doctors can work at most on c vacation days. The capacity of one from doctors to intermediate vertices enforces that each doctor works on at most one day of each period.

Proof: Flow implies assignment, and assignment implies flow.

- **Flow implies assignment:** If a maximum flow of size $\left|\bigcup_j D_j\right|$ exists:
  - Each vacation day d_k is assigned to exactly one doctor: The flow through the edge (d_k, t) is 1 for all k, ensuring that every day is covered.
  - No doctor is assigned more than c days total: The flow capacity of c on edges (s, u_i) ensures that no doctor works more than their allowed limit.
  - No doctor works more than one day in any period D_j: The intermediate nodes v_{ij} enforce this constraint through their edge capacities.
- **Assignment implies flow:** If a valid assignment of doctors to vacation days exists:
  - For each vacation day d_k, assign a unit of flow from the corresponding doctor node u_i through the intermediate node v_{ij} (if applicable) to d_k, and finally to the sink t.
  - The total flow is exactly $\left|\bigcup_j D_j\right|$, as each day is covered once.
  - The constraints on doctor availability, maximum workdays, and vacation periods are respected by construction, ensuring a valid flow.

[Extra Credit]

6. Prove one of the following problems is NP-hard.

- Given an undirected graph G, is it possible to color the vertices of G with three different colors, so that at most 31337 edges have both endpoints the same color?

  Solution: We show a reduction from the 3-coloring problem.

  Reduction from 3-coloring:

  (a) Given an instance $G = (V, E)$ of three coloring problem, we create an instance G' of the relaxed 3 coloring as follows.

  (b) To G add 31337 disjoint copies of a graph H, such that every 3 coloring of H has at least one edge with end point of same color. One such graph is K_4.

  (c) Run the algorithm to find if a 3-coloring with the relaxation that at most 31337 edges have both endpoints the same color exists.

  Proof of correctness: We show that G is 3 colorable if and only if G' is 3-colorable with at most 31337 edges have both endpoints the same color.

  (i) If G is 3-colorable, extended the coloring to G'. On the old vertices, use this coloring and for the copies of H, use any 3- coloring that colors at most one edge endpoints with the same color.

  (ii) If G' has a coloring with at most 31337 edge exceptions, then it must be that the all the exceptions occur on the copies of H, because any three coloring of H causes one edge exception, so we have at least 31337 exception from copies of H, and subgraph G cannot have any exceptions. Thus we have a genuine 3 coloring for the original graph G.

  Runtime analysis: This reduction can be implemented in $O(n)$ time, and hence a polynomial-time algorithm for this problem implies a polynomial-time algorithm for 3-coloring problem, which in turn implies P = NP (because 3-coloring is NP-hard). Thus, a solution for this problem solves any NP problem in polynomial time, making this problem NP-hard.

- Given an undirected graph $G = (V, E)$, does G contain a spanning tree in which every vertex has a degree of at most 7 ?

  Solution: Reduction (from Undirected s-t Hamiltonian Path):

  (a) Given an instance $G = (V, E)$ of the undirected s-t Hamiltonian path problem, we create an instance G' of the 7-degree spanning tree problem as follows.

  (b) For any vertex $v \in V \setminus \{s, t\}$, add 5 new dummy vertices to G' and connect them to v. Add 6 new dummy vertices for each of s and t, and connect them to s and t.

  (c) We run the algorithm for 7-degree spanning tree on G' and output that G has an s-t Hamiltonian path if and only if the algorithm outputs that G' has a 7-degree spanning tree.

  Proof of correctness: We show that G has an s-t Hamiltonian path if and only if G' has a 7-degree spanning tree.

  (i) If G has an s-t Hamiltonian path P, then G' has a 7-degree spanning tree:
  The tree in G' where each original vertex is connected to all its dummy vertices and its neighbor(s) in P is a spanning tree of G' where the degree of each vertex is at most 7.

  (ii) If G' has a spanning tree T in which every vertex has a degree of at most 7, then G has an s-t Hamiltonian path:
  Since the dummy vertices can only be connected to their corresponding original vertex, removing them from T does not change the connectivity of the remaining (original) vertices in T. Moreover, after removing them, the degree of each of s and t is exactly 1, and the remaining vertices have degree 2 in this new tree. Hence, the tree is a Hamiltonian path between s and t in G.

  Runtime analysis: This reduction can be implemented in $O(n+m)$ time, and hence a polynomial-time algorithm for 7-degree spanning tree implies a polynomial-time algorithm for the undirected s-t Hamiltonian path, which in turn implies P = NP (by definition of the s-t Hamiltonian path being NP-hard). Thus, a polynomial-time algorithm for the 7-degree spanning tree also implies P = NP, making this problem NP-hard.

- Given an undirected graph $G = (V, E)$, what is the size of the largest pyramid in G ? A pyramid is a set of vertices $v_1, \ldots, v_k$ plus another vertex u such that there are no edges between $v_1, \ldots, v_k$ in G and every vertex v_i is neighbor to u.

  Solution: Reduction (from Maximum Independent Set):

  (a) Given an instance $G = (V, E)$ of the maximum independent set problem, we create an instance G' of the maximum pyramid problem as follows.

  (b) Add a new vertex z to G and connect it to all vertices of G to obtain G'.

  (c) We run the algorithm for the maximum pyramid problem on G' to obtain its answer on G', denoted by k; we then output $k - 1$ as the answer to the maximum independent set problem in G.

  Proof of correctness: We show that for any integer t, G' has a pyramid of size t if and only if G has an independent set of size $t - 1$. This then implies that the maximum pyramid size in G' is always exactly one larger than the maximum independent set size in G, thus proving the correctness of the reduction.

  (i) If G has a maximum independent set of size t, then G' has a pyramid of size $t + 1$:
  Let $v_1, \ldots, v_t$ be the independent set in G. Consider the pyramid consisting of vertices $v_1, \ldots, v_t$ in G' plus the vertex z. There are no edges between $v_1, \ldots, v_t$ in G', and they are all neighbors to z. Thus, this is a valid pyramid, and its size is $t + 1$ as desired.

  (ii) If G' has a pyramid of size $t + 1$, then G has a maximum independent set of size t:
  Let $v_1, \ldots, v_t$ and u be the pyramid in G'. Note that since z has an edge to every vertex of the graph, none of the v_i's can be equal to z, and thus those vertices are all originally in G. Since there are no edges between $v_1, \ldots, v_t$, they form an independent set of size t in G as desired.
  (We emphasize that the pyramid found here may or may not have $u = z$, unlike part (i) where the top vertex of the pyramid was always z. However, this does not affect the correctness of the reduction.)

  Runtime analysis: This reduction can be implemented in $O(n+m)$ time, and hence a polynomial-time algorithm for the Maximum Pyramid Problem implies a polynomial-time algorithm for the Maximum Independent Set, which in turn implies P = NP (by definition of the Maximum Independent Set being NP-hard). Thus, a polynomial-time algorithm for the Maximum Pyramid Problem also implies P = NP, making this problem NP-hard.